

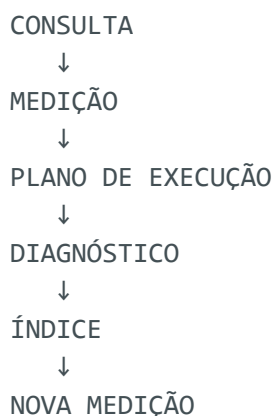
Roteiro de Laboratório — Performance Tuning no SQL Server

1. Objetivo

Demonstrar, em uma tabela de grande volume de dados fiscais, como o SQL Server executa consultas sem índices e como diferentes estratégias de indexação podem alterar:

- tempo de execução;
- consumo de CPU;
- **logical reads**;
- **physical reads**;
- quantidade de linhas processadas;
- operadores do plano de execução;
- **Table Scan**;
- **Index Scan**;
- **Index Seek**;
- **RID Lookup**;
- **Key Lookup**;
- **Sort**;
- **Hash Match**;
- custo de manutenção dos índices.

A metodologia será:



2. Banco e tabela utilizada

```
USE [933000081200000173202662_20260127_104141];  
GO
```

Tabela:

```
fisc.TB_202_PR_XML_ITENS_NOTAS_FISCAIS_PROPRIAS_E_TERCEIROS_01
```

Essa tabela representa itens de notas fiscais. Uma única NF-e pode possuir diversos itens:

```
1 NF-e  
  |  
  | item 1  
  | item 2  
  | item 3  
  | ...  
  | item N
```

Portanto, a tabela de itens tende a possuir cardinalidade muito maior que uma tabela de cabeçalho de NF-e.

Característica importante

O DDL apresentado não possui:

- PRIMARY KEY;
- UNIQUE;
- CLUSTERED INDEX;
- NONCLUSTERED INDEX.

O cenário inicial será, portanto, tratado como uma tabela **heap**.

3. Etapa 1 — Conhecer o volume da tabela

Comece medindo a quantidade de registros:

```
SELECT
    COUNT_BIG(*) AS quantidade_registros
FROM fisc.TB_202_PR_XML_ITENS_NOTAS_FISCAIS_PROPRIAS_E_TERCEIROS_01;
```

Depois:

```
EXEC sp_spaceused
'fisc.TB_202_PR_XML_ITENS_NOTAS_FISCAIS_PROPRIAS_E_TERCEIROS_01';
```

Registre os resultados

Informação	Resultado
Quantidade de registros	
Espaço reservado	
Espaço de dados	
Espaço de índices	

Objetivo

Demonstrar que o custo de uma consulta depende também do volume de dados que precisa ser processado.

4. Etapa 2 — Verificar os índices existentes

Execute:

```
EXEC sp_helpindex  
    'fisc.TB_202_PR_XML_ITENS_NOTAS_FISCAIS_PROPRIAS_E_TERCEIROS_01';
```

Registre os índices existentes.

Se o resultado confirmar que não há índices, o cenário será:

```
HEAP  
|  
▼  
Table Scan
```

Questão para discussão

Por que o SQL Server precisa percorrer a tabela quando não existe uma estrutura de acesso adequada para a consulta?

5. Etapa 3 — Criar a consulta pesada

A primeira consulta fará uma análise fiscal agrupando os dados por:

- ano;
- mês;
- UF do emitente;
- CFOP.

Além disso, serão calculados diversos indicadores.

```
SELECT  
    NF_ANO,  
    NF_MES,  
    NF_UF_EMITENTE,  
    ITEMNF_CFOP,  
  
    COUNT_BIG(*) AS quantidade_itens,  
  
    COUNT(DISTINCT NF_CHAVE_ACESSO) AS quantidade_notas,  
  
    SUM(ITEMNF_VPROD) AS valor_produtos,  
  
    SUM(ITEMNF_VDESC) AS valor_desconto,
```

```

SUM(ITEMNF_VFRETE) AS valor_frete,

SUM(ITEMNF_VSEG) AS valor_seguro,

SUM(ITEMNF_VOUTRO) AS outras_despesas,

SUM(ITEMNF_VBC) AS base_icms,

SUM(ITEMNF_VLICMS) AS valor_icms,

SUM(ITEMNF_VBCST) AS base_icms_st,

SUM(ITEMNF_VICMSST) AS valor_icms_st,

SUM(ITEMNF_VBC_IPI) AS base_ipi,

SUM(ITEMNF_VIPI) AS valor_ipi,

AVG(ITEMNF_VPROD) AS valor_medio_item,

MIN(ITEMNF_VPROD) AS menor_item,

MAX(ITEMNF_VPROD) AS maior_item

FROM fisc.TB_202_PR_XML_ITENS_NOTAS_FISCAIS_PROPRIAS_E_TERCEIROS_01

GROUP BY
    NF_ANO,
    NF_MES,
    NF_UF_EMITENTE,
    ITEMNF_CFOP

ORDER BY
    NF_ANO,
    NF_MES,
    NF_UF_EMITENTE,
    ITEMNF_CFOP;

```

A consulta combina:

```

COUNT_BIG
COUNT(DISTINCT)
SUM
AVG
MIN
MAX
GROUP BY
ORDER BY

```

sobre uma tabela potencialmente muito grande.

6. Etapa 4 — Medir o cenário inicial

Ative as estatísticas:

```
SET STATISTICS IO ON;  
SET STATISTICS TIME ON;
```

Execute a consulta.

Depois desative:

```
SET STATISTICS IO OFF;  
SET STATISTICS TIME OFF;
```

Registre:

Métrica	Sem índice
CPU time	
Elapsed time	
Logical reads	
Physical reads	
Operador principal	
Memória	

7. Etapa 5 — Capturar o plano de execução

No SQL Server Management Studio:

```
Ctrl + M
```

Execute novamente a consulta.

Como a tabela é um heap, é esperado encontrar algum operador semelhante a:

```
Table Scan
```

Uma representação conceitual:

```
Table Scan
  ↓
Compute Scalar
  ↓
Hash Match / Aggregate
  ↓
Sort
  ↓
SELECT
```

Questão principal

Por que o SQL Server precisou ler tantas linhas?

O objetivo não é apenas identificar o operador mais caro, mas compreender **por que o otimizador escolheu aquele plano**.

8. Etapa 6 — Criar o primeiro índice

Crie um índice sobre a data de emissão:

```
CREATE INDEX IX_TB202_NF_DHEMI
ON fisc.TB_202_PR_XML_ITENS_NOTAS_FISCAIS_PROPRIAS_E_TERCEIROS_01
(
    NF_DHEMI
);
```

Verifique:

```
EXEC sp_helpindex
    'fisc.TB_202_PR_XML_ITENS_NOTAS_FISCAIS_PROPRIAS_E_TERCEIROS_01';
```

9. Etapa 7 — Criar uma consulta seletiva

Agora vamos analisar apenas um período.

Exemplo: janeiro de 2026.

```
SELECT
    COUNT_BIG(*) AS quantidade_itens,

    COUNT(DISTINCT NF_CHAVE_ACESSO) AS quantidade_notas,

    SUM(ITEMNF_VPROD) AS valor_produtos,

    SUM(ITEMNF_VDESC) AS valor_desconto,

    SUM(ITEMNF_VLICMS) AS valor_icms,

    SUM(ITEMNF_VICMSST) AS valor_icms_st,

    SUM(ITEMNF_VIPI) AS valor_ipi

FROM fisc.TB_202_PR_XML_ITENS_NOTAS_FISCAIS_PROPRIAS_E_TERCEIROS_01

WHERE NF_DHEMI >= '20260101'
      AND NF_DHEMI < '20260201';
```

Agora o SQL Server pode utilizar o índice para procurar diretamente o intervalo solicitado.

Conceitualmente:

```
IX_TB202_NF_DHEMI
  |
  ▼
Index Seek
  |
  ▼
somente janeiro/2026
```

10. Etapa 8 — Comparar Table Scan × Index Seek

A comparação entre **Table Scan** e **Index Seek** é central para entender o comportamento do otimizador. A ideia é simples: quando a consulta precisa localizar um subconjunto pequeno de linhas, a busca direta em um índice geralmente é mais eficiente do que percorrer toda a tabela. Quando a consulta exige praticamente todos os registros, um **Table Scan** pode ser até mais vantajoso do que buscar em um índice, porque o custo de navegação no índice pode ser maior do que o custo de varrer a estrutura de dados inteira.

Exemplo 1 — consulta sem índice

Se a tabela ainda estiver em estado de **heap** e a consulta for:

```
SELECT
    COUNT_BIG(*) AS quantidade_itens,
    SUM(ITEMNF_VPROD) AS valor_produtos
FROM fisc.TB_202_PR_XML_ITENS_NOTAS_FISCAIS_PROPRIAS_E_TERCEIROS_01
WHERE NF_DHEMI >= '20260101'
      AND NF_DHEMI < '20260201';
```

e não houver índice em **NF_DHEMI**, o SQL Server não consegue localizar rapidamente somente o período pesquisado. Em vez disso, ele percorre a tabela inteira.

Conceitualmente:

```
HEAP
|
▼
Table Scan
|
▼
toda a tabela
|
▼
filtro de data
```

Esse operador representa uma leitura sequencial de grande parte da estrutura. O custo cresce conforme o volume de dados aumenta, principalmente em tabelas de itens fiscais.

Exemplo 2 — consulta com índice em data

Agora, depois de criar o índice:

```
CREATE INDEX IX_TB202_NF_DHEMI
ON fisc.TB_202_PR_XML_ITENS_NOTAS_FISCAIS_PROPRIAS_E_TERCEIROS_01
(
    NF_DHEMI
);
```

a mesma consulta pode ser atendida por um caminho mais direto:

```
graph TD
    A[IX_TB202_NF_DHEMI] --> B[Index Seek]
    B --> C[somente registros do período]
```

Nesse caso, o SQL Server utiliza a árvore do índice para localizar rapidamente o intervalo de datas solicitado e evitar a leitura de todo o conjunto de registros.

Quando aparece cada operador?

Table Scan

Aparece quando o otimizador decide percorrer a tabela inteira, geralmente porque:

- não há índice útil para o predicado;
- o filtro retorna uma parte grande da tabela;
- a análise exige a leitura de muitos registros;
- o custo estimado do scan é menor do que o custo de usar o índice.

Index Seek

Aparece quando o otimizador consegue localizar diretamente os registros relevantes usando a estrutura de índice, especialmente quando:

- a coluna do filtro é indexada;
- o predicado é seletivo;
- a condição é SARGable, como `>=` e `<` em intervalos de datas;
- o índice permite identificar um subconjunto pequeno dos dados.

Comparação prática

Cenário A — filtro por data em uma tabela grande

```
WHERE NF_DHEMI >= '20260101'  
AND NF_DHEMI < '20260201';
```

Isso normalmente favorece **Index Seek**, porque o período é uma fração pequena da tabela.

Cenário B — consulta sem filtro ou filtro muito amplo

```
WHERE NF_UF_EMITENTE = 'SP';
```

se a UF tiver muitos registros, o otimizador pode preferir um **Table Scan** ou um **Index Scan**, dependendo da seletividade e do número estimado de linhas.

O que observar no plano

No plano de execução, é importante olhar:

- operador de acesso: **Table Scan** ou **Index Seek**;
- **Estimated Number of Rows**;
- **Actual Number of Rows**;

- **Logical Reads**;
- custo estimado do operador.

Se o plano mostra **Table Scan** para uma consulta altamente seletiva, isso costuma indicar que:

- não existe um índice adequado;
- o filtro não está **SARGable**;
- o otimizador calculou que a leitura completa pode ser mais barata que a busca indexada.

Conclusão

Table Scan e **Index Seek** representam duas formas diferentes de localizar dados:

- **Table Scan** é uma leitura ampla e mais simples;
- **Index Seek** é uma busca direcionada e mais eficiente quando há seletividade.

A escolha correta depende do padrão de acesso, da seletividade, do volume de dados e do plano estimado pelo otimizador.

Questão para discussão

Em uma tabela fiscal com milhões de itens, qual operador tende a ser mais eficiente para um filtro por data: **Table Scan** ou **Index Seek**? Justifique com base na seletividade da consulta.

11. Etapa 9 — Demonstrar SARGability

Uma condição é considerada **SARGable** quando ela permite ao otimizador do SQL Server usar um índice de forma eficiente para localizar as linhas relevantes. O termo vem de **Search Argument Able**, ou seja, a expressão pode servir como argumento de busca útil ao índice.

Em outras palavras, uma condição **SARGable** "fala diretamente" com a coluna indexada. Quando a coluna é transformada, convertida ou usada dentro de funções, o

índice deixa de ser tão útil e o otimizador tende a escolher uma estratégia mais custosa, como **Table Scan** ou **Index Scan**.

Consulta A — usando funções sobre a coluna

```
SELECT
    COUNT_BIG(*) AS quantidade_itens,
    SUM(ITEMNF_VPROD) AS valor_produtos
FROM fisc.TB_202_PR_XML_ITENS_NOTAS_FISCAIS_PROPRIAS_E_TERCEIROS_01
WHERE YEAR(NF_DHEMI) = 2026
    AND MONTH(NF_DHEMI) = 1;
```

Essa condição não é SARGable, porque o SQL Server precisa aplicar **YEAR()** e **MONTH()** sobre a coluna antes da comparação. Isso reduz a capacidade do índice em **NF_DHEMI** de ser aproveitado de maneira eficiente.

Consulta B — utilizando intervalo

```
SELECT
    COUNT_BIG(*) AS quantidade_itens,
    SUM(ITEMNF_VPROD) AS valor_produtos
FROM fisc.TB_202_PR_XML_ITENS_NOTAS_FISCAIS_PROPRIAS_E_TERCEIROS_01
WHERE NF_DHEMI >= '20260101'
    AND NF_DHEMI < '20260201';
```

Essa condição é SARGable, porque a comparação é feita diretamente sobre a coluna indexada. O otimizador consegue localizar o intervalo de datas de forma mais eficiente e, normalmente, usa um **Index Seek** em vez de percorrer toda a tabela.

Exemplo adicional de condição não SARGable

```
WHERE CONVERT(VARCHAR(10), NF_DHEMI, 112) = '20260101';
```

Aqui, a função `CONVERT()` atua sobre a coluna antes da comparação, o que geralmente impede o uso eficiente do índice. A mesma lógica vale para condições como:

```
WHERE LTRIM(RTRIM(NF_UF_EMITENTE)) = 'SP';
```

Se a intenção é preservar a capacidade de busca pelo índice, o ideal é evitar transformações diretas sobre a coluna no predicado.

Por que isso é importante no laboratório

Ao comparar os planos de execução:

- uma condição SARGable tende a produzir `Index Seek`;
- uma condição não SARGable tende a gerar `Table Scan` ou `Index Scan`;
- o custo em `logical reads` e `CPU` costuma aumentar no segundo caso.

Conclusão

A regra prática é simples:

- compare a coluna diretamente;
- evite converter, transformar ou aplicar funções sobre a coluna no `WHERE`;
- prefira intervalos e comparações simples quando o objetivo for usar o índice de forma eficiente.

Execute ambas com:

```
SET STATISTICS IO ON;  
SET STATISTICS TIME ON;
```

Depois compare:

- plano de execução;
- `logical reads`;
- CPU;
- tempo total;

- operador de acesso.

Conceito

A primeira consulta aplica funções sobre a coluna:

```
YEAR(NF_DHEMI)
MONTH(NF_DHEMI)
```

A segunda trabalha diretamente com a coluna:

```
NF_DHEMI >= ...
AND NF_DHEMI < ...
```

A segunda forma é mais favorável à utilização eficiente de índices e é um exemplo clássico de uma condição **SARGable**.

12. Etapa 10 — Criar um índice de cobertura

Até aqui, o índice simples em **NF_DHEMI** já ajuda o SQL Server a localizar rapidamente os registros de um período. Porém, em muitas consultas analíticas, a busca por data é apenas o ponto de partida. Em seguida, o mecanismo precisa também de outras colunas para calcular agregados, como **NF_CHAVE_ACESSO**, **ITEMNF_VPROD**, **ITEMNF_VDESC**, **ITEMNF_VLICMS**, **ITEMNF_VICMSST** e **ITEMNF_VIPI**.

Esse é o ponto em que o índice simples pode deixar de ser suficiente. Em um índice simples, o banco geralmente guarda a chave do índice e o identificador da linha. Se a consulta precisa de outras colunas além da chave, ele pode ter que ir até a tabela base para buscar o restante dos dados. Isso gera o que chamamos de **Lookup** adicional.

Um índice de cobertura é diferente: ele foi criado para que a consulta consiga obter todas as colunas que ela precisa diretamente do próprio índice, sem precisar acessar o heap ou a tabela de dados principal. Em outras palavras, o SQL Server consegue

responder a consulta usando apenas o índice, o que reduz leitura extra, reduz custo de I/O e pode melhorar bastante o desempenho em relatórios e consultas agregadas.

Diferença entre índice simples e índice de cobertura

Índice simples

```
CREATE INDEX IX_TB202_NF_DHEMI
ON fisc.TB_202_PR_XML_ITENS_NOTAS_FISCAIS_PROPRIAS_E_TERCEIROS_01
(
    NF_DHEMI
);
```

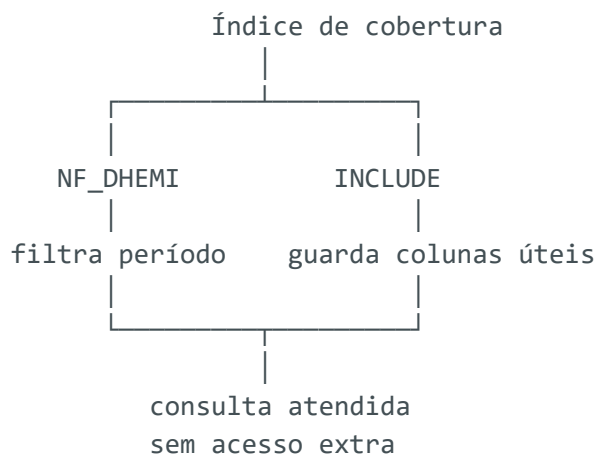
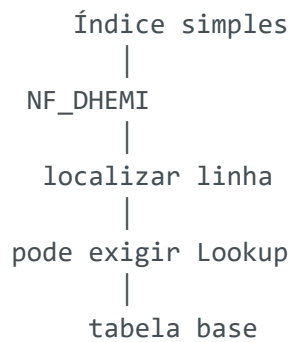
Esse índice organiza a busca pela data. Ele é muito útil quando a consulta filtra por período e a chave do índice já resolve bem a condição. Mas, se a consulta também exigir outras colunas, o otimizador pode precisar buscar os dados completos em outra estrutura.

Índice de cobertura

```
CREATE INDEX IX_TB202_NF_DHEMI_COVERING
ON fisc.TB_202_PR_XML_ITENS_NOTAS_FISCAIS_PROPRIAS_E_TERCEIROS_01
(
    NF_DHEMI
)
INCLUDE
(
    NF_CHAVE_ACESSO,
    ITEMNF_VPROD,
    ITEMNF_VDESC,
    ITEMNF_VLICMS,
    ITEMNF_VICMSST,
    ITEMNF_VIPI
);
```

Esse índice mantém a busca por **NF_DHEMI** como chave e, além disso, armazena as colunas usadas no cálculo e no filtro. Com isso, o SQL Server consegue atender a consulta sem procurar os dados na tabela física. Ele funciona como um "resultado parcial pronto para consumo".

Conceitualmente:



Nesse caso, o objetivo é reduzir a necessidade de buscar dados adicionais na estrutura principal.

Aplicabilidade

O índice de cobertura tem grande aplicação em consultas que:

- filtram por um intervalo de datas ou outra chave seletiva;
- calculam agregações (**SUM**, **AVG**, **COUNT**, **MIN**, **MAX**);
- acessam poucas colunas em comparação com a tabela inteira;
- são executadas frequentemente em relatórios, dashboards e auditorias.

Ele é especialmente útil quando a consulta não precisa de muitas colunas da tabela, mas exige uma combinação de filtro e projeção. Em cenários de análise fiscal, por exemplo, um índice em **NF_DHEMI** com **INCLUDE** das colunas monetárias e de chave de documento pode reduzir bastante a carga de leitura.

Atenção importante

O índice de cobertura não é uma solução universal. Ele pode aumentar:

- o espaço em disco necessário;
- o custo de manutenção em **INSERT**, **UPDATE** e **DELETE**;
- a quantidade de dados que precisam ser regravados quando o índice muda.

Logo, ele deve ser usado quando a consulta frequente justificar a vantagem em desempenho. Em outras palavras, o benefício aparece quando a economia de leitura e de **Lookup** compensa o custo adicional de manter o índice.

Questão para discussão

Em uma consulta de auditoria fiscal com filtro por data e vários campos agregados, vale a pena um índice de cobertura? Quando o ganho real fica mais evidente?

13. Etapa 11 — Procurar RID Lookup

Observe o plano de execução.

Como a tabela original é um heap, pode aparecer:

RID Lookup

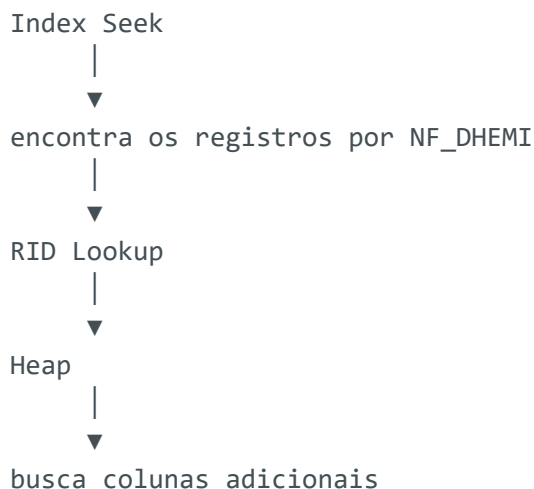
Esse operador indica que o SQL Server fez uma busca no índice para localizar as linhas relevantes, mas, para concluir a consulta, precisou voltar à tabela base para obter colunas que não estavam presentes no índice. Em um heap, a referência de localização da linha é o RID, que funciona como um identificador do registro físico dentro da tabela.

Em outras palavras, o banco fez algo como:

1. localiza as linhas no índice pela data;
2. obtém o RID;

3. usa esse RID para ir até o heap;
4. busca os demais campos que a consulta precisa.

Uma representação conceitual:



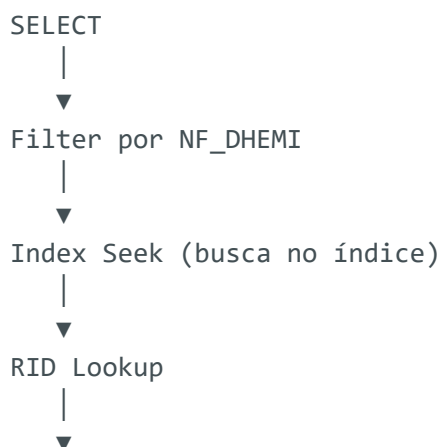
Isso significa que o plano de execução é composto por duas etapas:

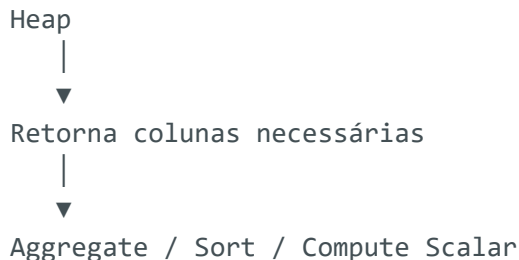
- uma busca seletiva no índice para reduzir o volume de linhas;
- uma volta à tabela para completar os dados que o índice não contém.

Esse comportamento é típico de consultas em heap quando o índice não é de cobertura. O custo do **RID Lookup** pode ser elevado, porque o SQL Server precisa acessar a estrutura de dados principal para cada conjunto de linhas identificadas pela chave do índice.

Como o plano de execução é montado

Quando a consulta filtra por data e depois precisa de valores monetários, descrições e chaves de documento, o otimizador pode montar um plano nessa ordem:





O ponto crítico é que o **Index Seek** reduz a quantidade de linhas a serem lidas, mas o **RID Lookup** pode anular parte do ganho, porque a consulta ainda precisa acessar o heap. Esse é um dos motivos pelos quais um índice de cobertura pode ser mais eficiente: ele guarda as colunas úteis junto com a chave e reduz ou elimina esse acesso extra.

Questão

Qual é o custo de utilizar um índice que encontra a linha, mas não possui todas as informações necessárias para produzir o resultado?

Observação prática

Se a consulta é muito frequente e o conjunto de colunas acessadas é estável, o índice de cobertura costuma produzir uma vantagem maior do que um índice simples. Já em consultas muito pontuais ou em tabelas com baixa cardinalidade, o ganho pode ser menos relevante, e o custo extra de manutenção do índice pode não compensar.

14. Etapa 12 — Criar um índice composto

Agora vamos testar um índice alinhado às dimensões da consulta analítica:

```
CREATE INDEX IX_TB202_PERIODO_UF_CFOP
ON fisc.TB_202_PR_XML_ITENS_NOTAS_FISCAIS_PROPRIAS_E_TERCEIROS_01
(
    NF_ANO,
    NF_MES,
    NF_UF_EMITENTE,
    ITEMNF_CFOP
);
```

O índice está alinhado com:

```
GROUP BY
  NF_ANO,
  NF_MES,
  NF_UF_EMITENTE,
  ITEMNF_CFOP
```

Discussão

Um índice deve ser projetado considerando os **padrões reais de acesso aos dados**, e não apenas porque determinada coluna existe na tabela.

15. Etapa 13 — Comparar as estratégias

Ao final, teremos quatro cenários experimentais.

Cenário 1 — Heap

Table Scan

Cenário 2 — Índice em data

IX_TB202_NF_DHEMI

Index Seek / Index Scan

Cenário 3 — Índice de cobertura

IX_TB202_NF_DHEMI_COVERING

Index Seek

+
menor necessidade de Lookup

Cenário 4 — Índice composto

IX_TB202_PERIODO_UF_CFOP

16. Etapa 14 — Montar a tabela de resultados

Preencha durante o laboratório:

Cenário	Índice	Operador	Logical Reads	CPU	Tempo
1	Nenhum	Table Scan			
2	NF_DHEMI				
3	Covering				
4	Composto				

Análise

O aluno deverá explicar:

1. Qual cenário apresentou maior quantidade de logical reads?
2. Qual apresentou menor tempo?
3. Qual apresentou menor consumo de CPU?
4. O plano mudou?
5. O SQL Server utilizou Index Seek?
6. Houve RID Lookup?
7. O índice de cobertura trouxe benefício?
8. O índice composto foi utilizado?
9. O resultado da consulta permaneceu igual?

17. Etapa 15 — Demonstrar o custo dos índices

Índices não são gratuitos.

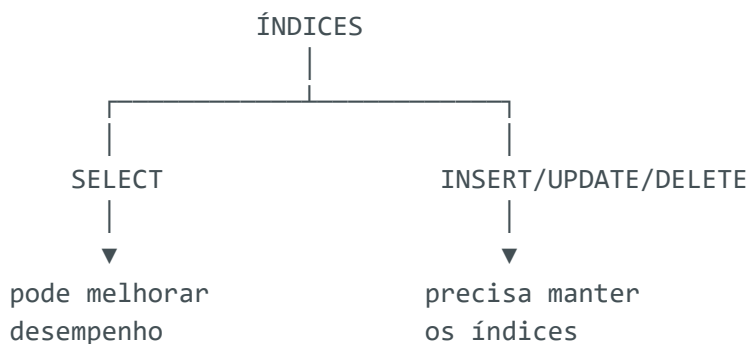
Eles podem melhorar:

SELECT

mas também introduzem custos em:

INSERT
UPDATE
DELETE

Conceitualmente:



Os custos incluem:

- espaço em disco;
 - memória;
 - CPU;
 - manutenção;
 - **INSERT**;
 - **UPDATE**;
 - **DELETE**;
 - **REBUILD**;
 - **REORGANIZE**.
-

18. Etapa 16 — Atualizar estatísticas

Depois das alterações:

```
UPDATE STATISTICS
fisc.TB_202_PR_XML_ITENS_NOTAS_FISCAIS_PROPRIAS_E_TERCEIROS_01;
```

Execute novamente as consultas.

Explique que o otimizador utiliza estatísticas para estimar:

quantas linhas serão retornadas?

Essa estimativa influencia decisões como:

```
Index Seek
Index Scan
Table Scan
Nested Loops
Hash Match
Merge Join
```

19. Etapa 17 — Comparar Estimated × Actual Rows

No plano de execução, observe:

```
Estimated Number of Rows
          VS
Actual Number of Rows
```

Exemplo hipotético:

```
Estimated: 10.000
Actual:    2.500.000
```

Uma diferença grande entre esses valores pode levar o otimizador a escolher um plano inadequado.

Questão

O que pode acontecer quando o SQL Server estima uma quantidade de linhas muito diferente da quantidade realmente processada?

20. Etapa 18 — Remover os índices experimentais

Ao finalizar o laboratório, remova os índices criados experimentalmente:

```
DROP INDEX IX_TB202_NF_DHEMI
ON fisc.TB_202_PR_XML_ITENS_NOTAS_FISCAIS_PROPRIAS_E_TERCEIROS_01;
```

```
DROP INDEX IX_TB202_NF_DHEMI_COVERING
ON fisc.TB_202_PR_XML_ITENS_NOTAS_FISCAIS_PROPRIAS_E_TERCEIROS_01;
```

```
DROP INDEX IX_TB202_PERIODO_UF_CFOP
ON fisc.TB_202_PR_XML_ITENS_NOTAS_FISCAIS_PROPRIAS_E_TERCEIROS_01;
```

Atenção: execute os comandos **DROP INDEX** somente se esses índices tiverem sido criados especificamente para o laboratório. Não remova índices existentes no ambiente de produção.

21. Sequência didática recomendada

```
01. Conhecer a tabela
    ↓
02. Medir quantidade de registros
    ↓
03. Verificar índices existentes
    ↓
```

04. Executar consulta pesada
↓
05. STATISTICS IO / TIME
↓
06. Plano de execução
↓
07. Identificar Table Scan
↓
08. Criar índice em NF_DHEMI
↓
09. Executar consulta seletiva
↓
10. Comparar Scan × Seek
↓
11. Demonstrar SARGability
↓
12. Criar índice de cobertura
↓
13. Analisar RID Lookup
↓
14. Criar índice composto
↓
15. Comparar estratégias
↓
16. Analisar estatísticas
↓
17. Comparar Estimated × Actual Rows
↓
18. Discutir custo dos índices
↓
19. Remover índices experimentais
↓
20. Conclusão

22. Conceitos que o aluno deverá dominar

Ao final do laboratório, o aluno deverá compreender:

- Heap;
- Table Scan;
- Clustered Index;
- Nonclustered Index;
- Index Scan;
- Index Seek;
- RID Lookup;
- Key Lookup;

- índice de cobertura;
- índice composto;
- **INCLUDE**;
- SARGability;
- **STATISTICS IO**;
- **STATISTICS TIME**;
- plano de execução;
- cardinalidade;
- estimativa de linhas;
- estatísticas;
- custo de manutenção dos índices.

23. Conclusão esperada

O objetivo principal do laboratório não é demonstrar que:

"Índice deixa a consulta rápida".

O objetivo é demonstrar o processo correto de **Performance Tuning**:

```
MEDIR
↓
OBSERVAR
↓
FORMULAR HIPÓTESE
↓
ALTERAR
↓
MEDIR NOVAMENTE
↓
COMPARAR
↓
VALIDAR
```

O SQL Server pode ou não utilizar o índice criado. Portanto, não devemos assumir antecipadamente que um índice produzirá **Index Seek** ou que será necessariamente mais rápido.

A decisão do otimizador depende de fatores como:

- cardinalidade;
- seletividade;

- distribuição dos dados;
- estatísticas;
- custo estimado;
- quantidade de páginas;
- memória disponível;
- predicados da consulta;
- índices disponíveis.

A evidência deve vir do plano de execução e das métricas de desempenho.